

GRAPH DATA PROCESSING PIPELINE IN PYTHON

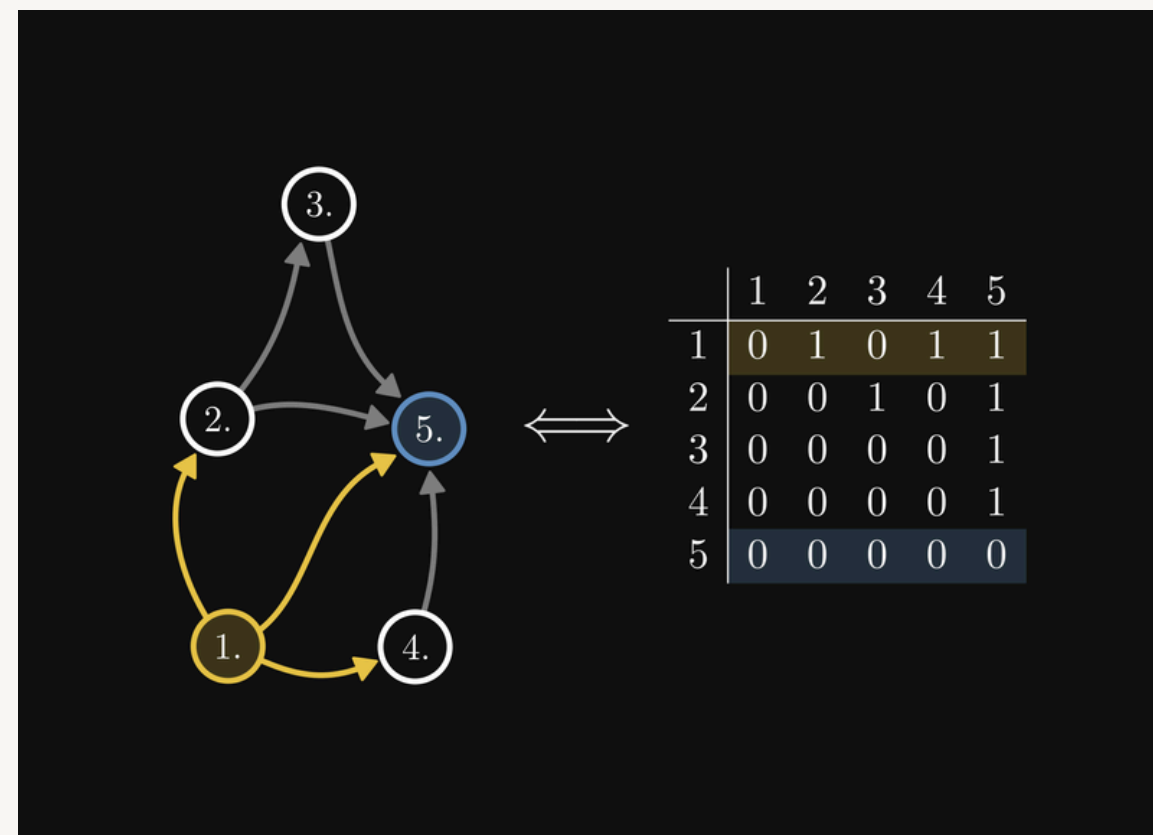
Recommendation score benchmark:

Graph DB vs chDB vs Pandas vs Polars vs NumPy

RESEARCH QUESTION

MAIN QUESTION

How does the same graph-based recommendation task behave in different Python data processing tools?



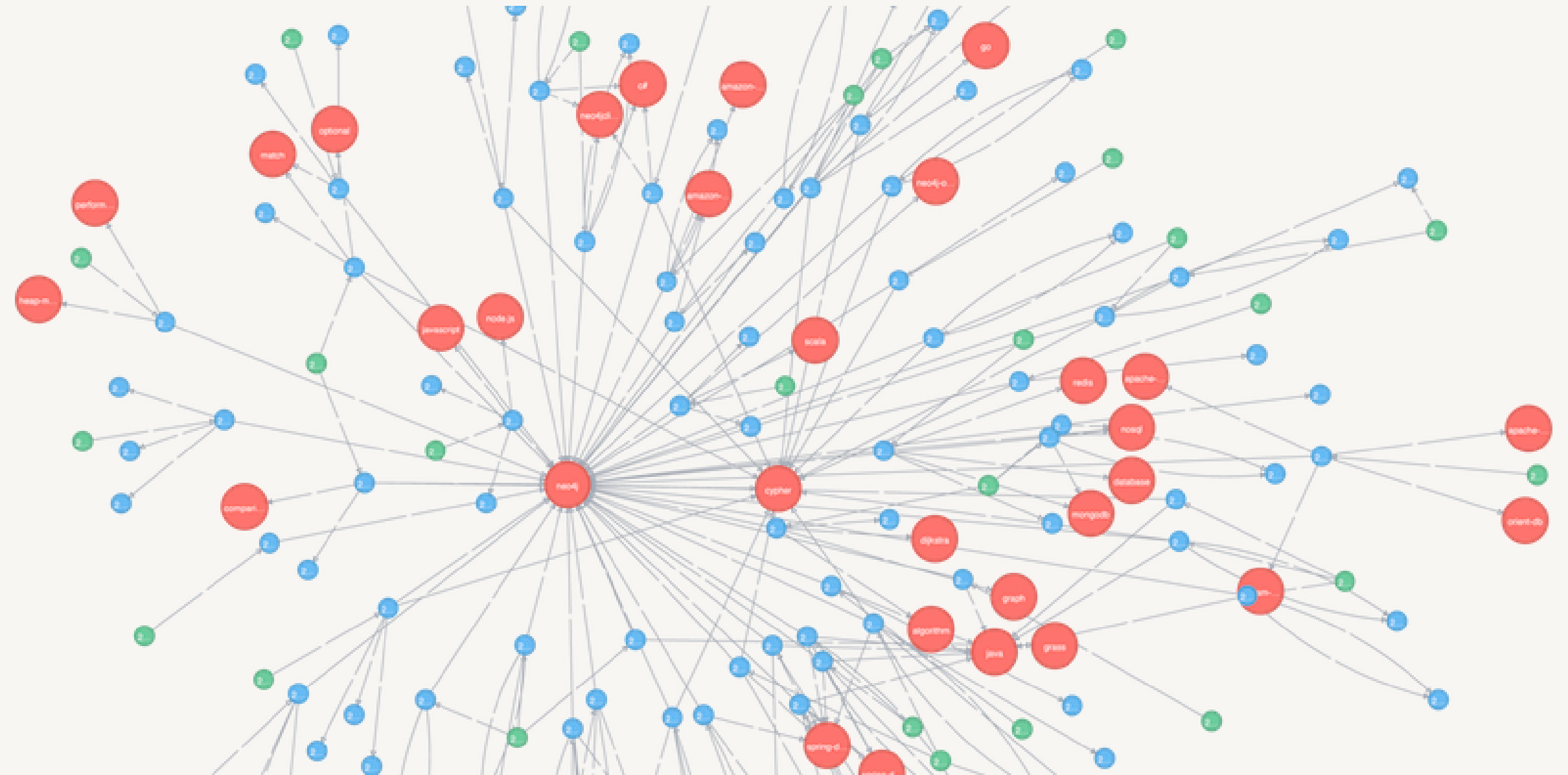
GRAPH DATABASES

stores data as:

- nodes
- relationships
- properties
- labels

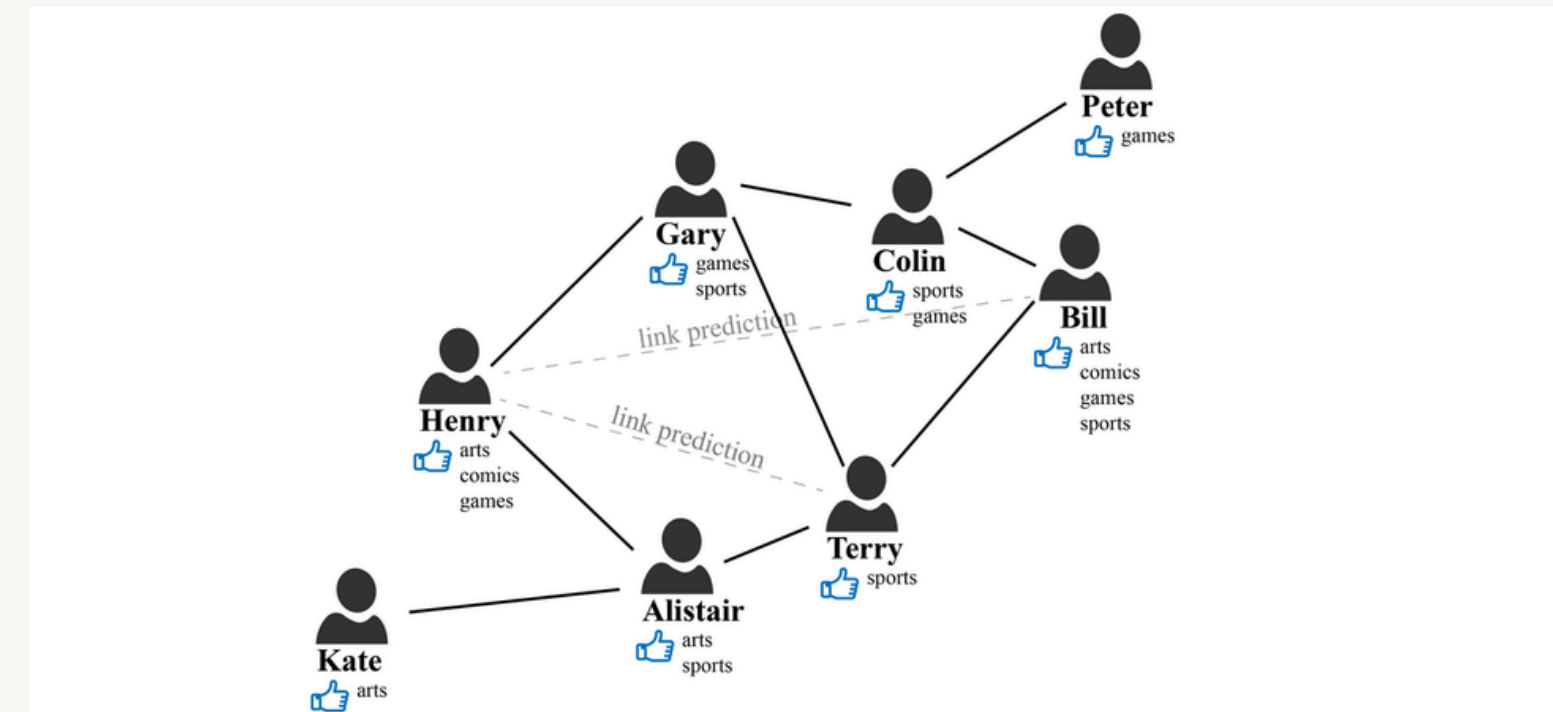
usage:

- recommendation systems
- fraud detection
- social network
- dependency analysis
- knowledge graphs



WHY RECOMMENDATIONS ARE GRAPH DATA

Graphs are usually used when relationships matter. A graph in recommendation systems represents users, items, and their interactions (like purchases or clicks) as nodes connected by edges. This structure uncovers hidden, multi-step relationships, allowing systems to provide highly accurate, explainable, and personalized suggestions.



RECOMMENDATION APPROACHES



- Common neighbors
- Jaccard Similarity
- Cosine Similarity

Mine recommendation logic:

User → Seen items → Similar users → Candidate items

score(item) = number of similar users

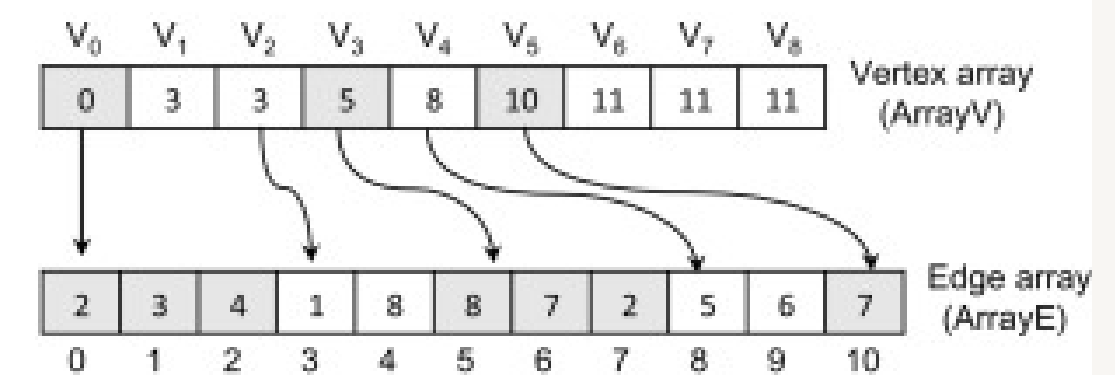
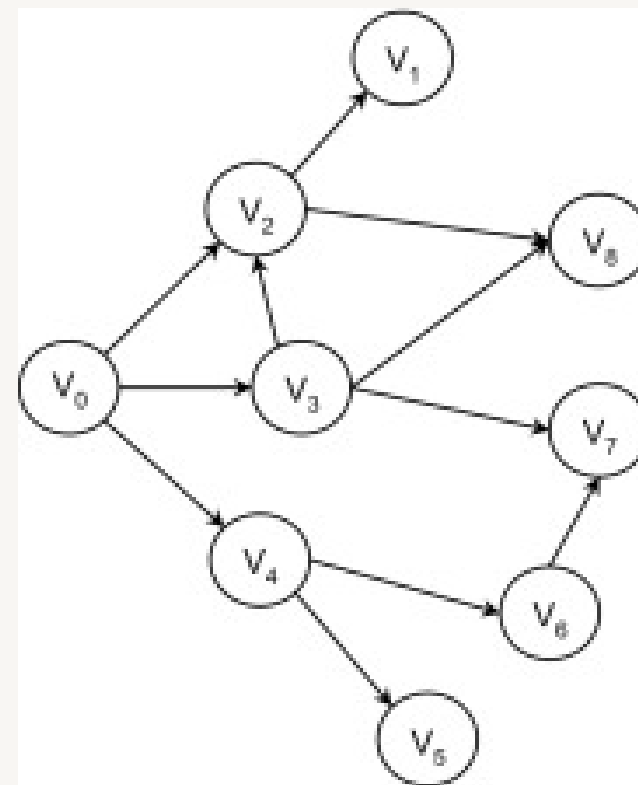
who interacted with this item

KUZU

Embedded Graph DB for analytical workloads

Under-the-hood:

- Columnar storage
- CSR
- vectorized execution



CHDB

chDB is a fast embedded in-process SQL
OLAP engine based on ClickHouse



GRAPH REPRESENTATION IN PYTHON

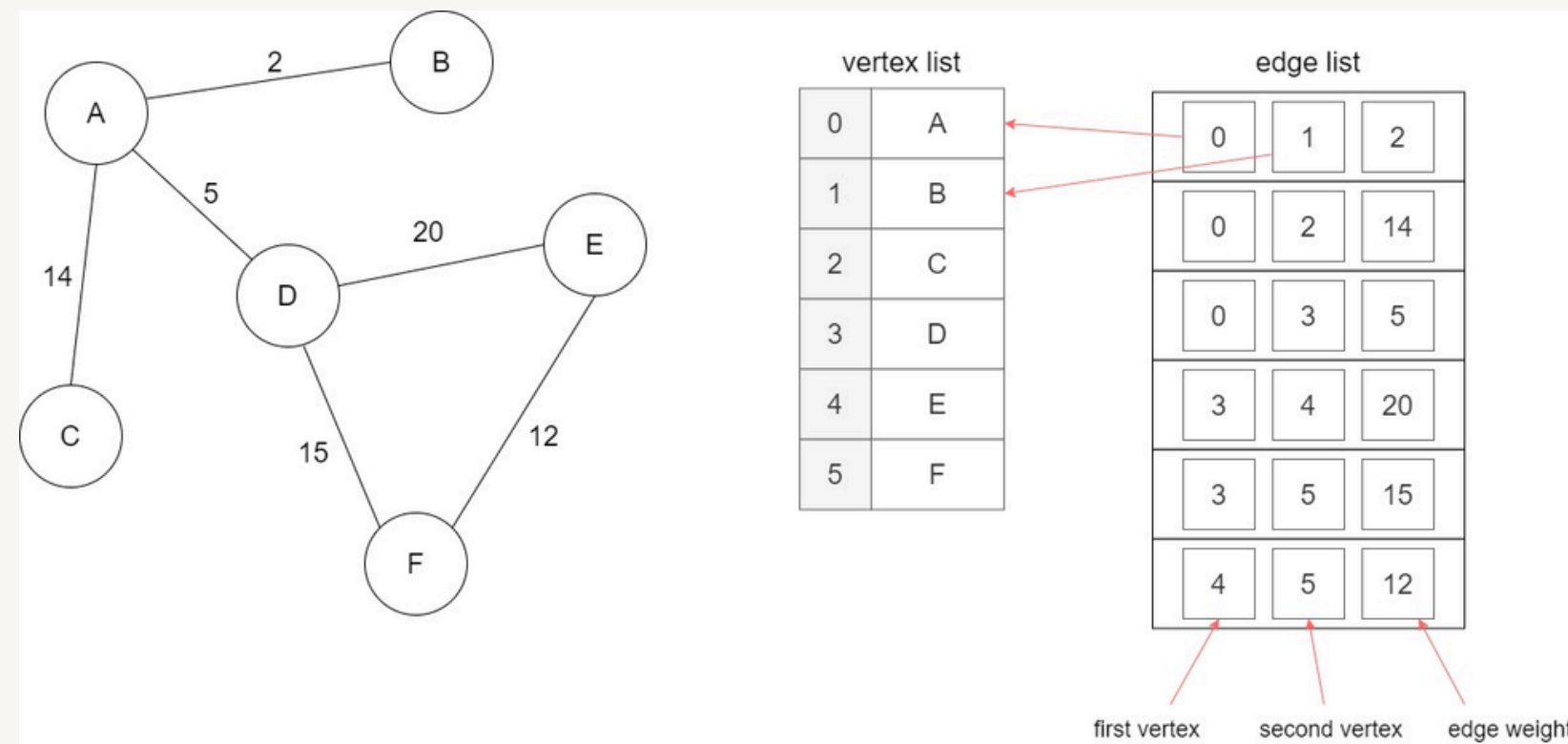
```
src = [user_id]
dst = [repo_id]
weight = [interaction_weight]
```

```
rows = users
columns = repositories
values = interactions
```

user_id	repo_id	event_type	weight	timestamp
1	101	star	1	1/1/2025
1	102	star	1	1/2/2025
2	101	star	1	1/3/2025

- edge list
- DataFrame
- NumPy arrays
- graph database
- SQL table

EDGE LIST



Each row stores one interaction between a user and an item.

NUMPY ARRAYS

Command

```
np.array([1,2,3])
```

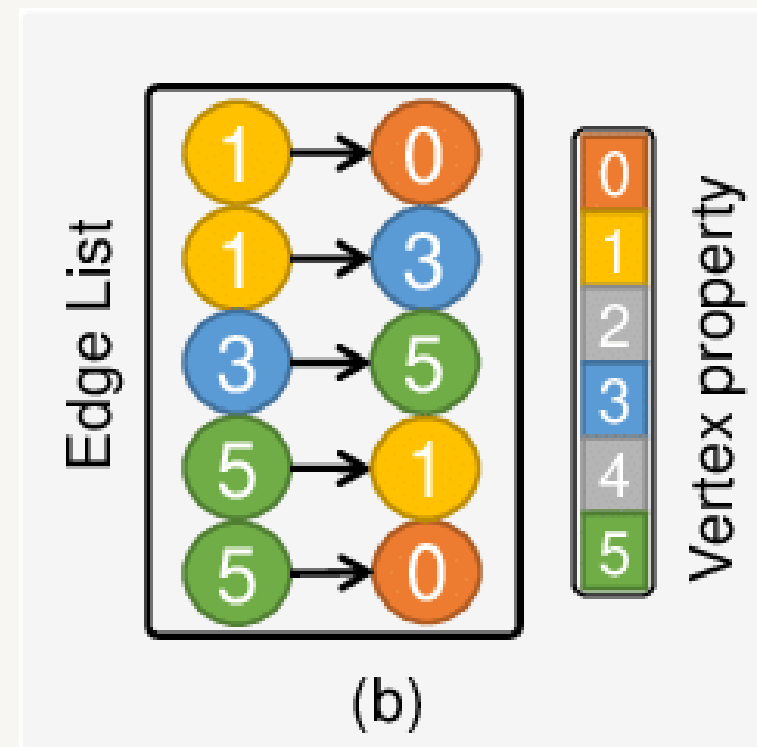
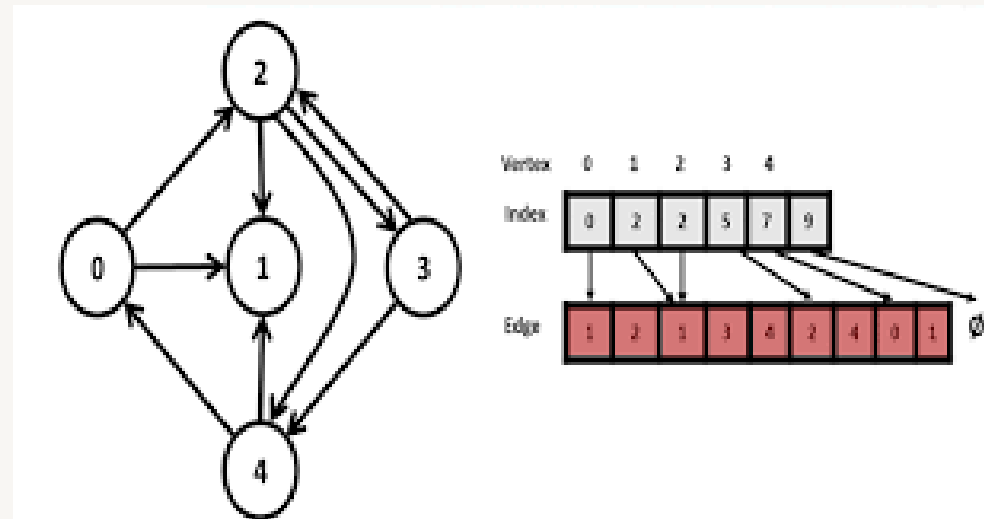


NumPy Array

1
2
3

Graph edges stored as arrays: source ids,
target ids and weights.

SPARSE MATRIX / CSR / COO



- A sparse matrix is useful when most values are zero.
- COO is useful for building the matrix from an edge list.
- CSR is useful for fast row access and matrix operations.

COMPARED TOOLS

Why tools differ?

Different execution models

PRACTICAL TASK

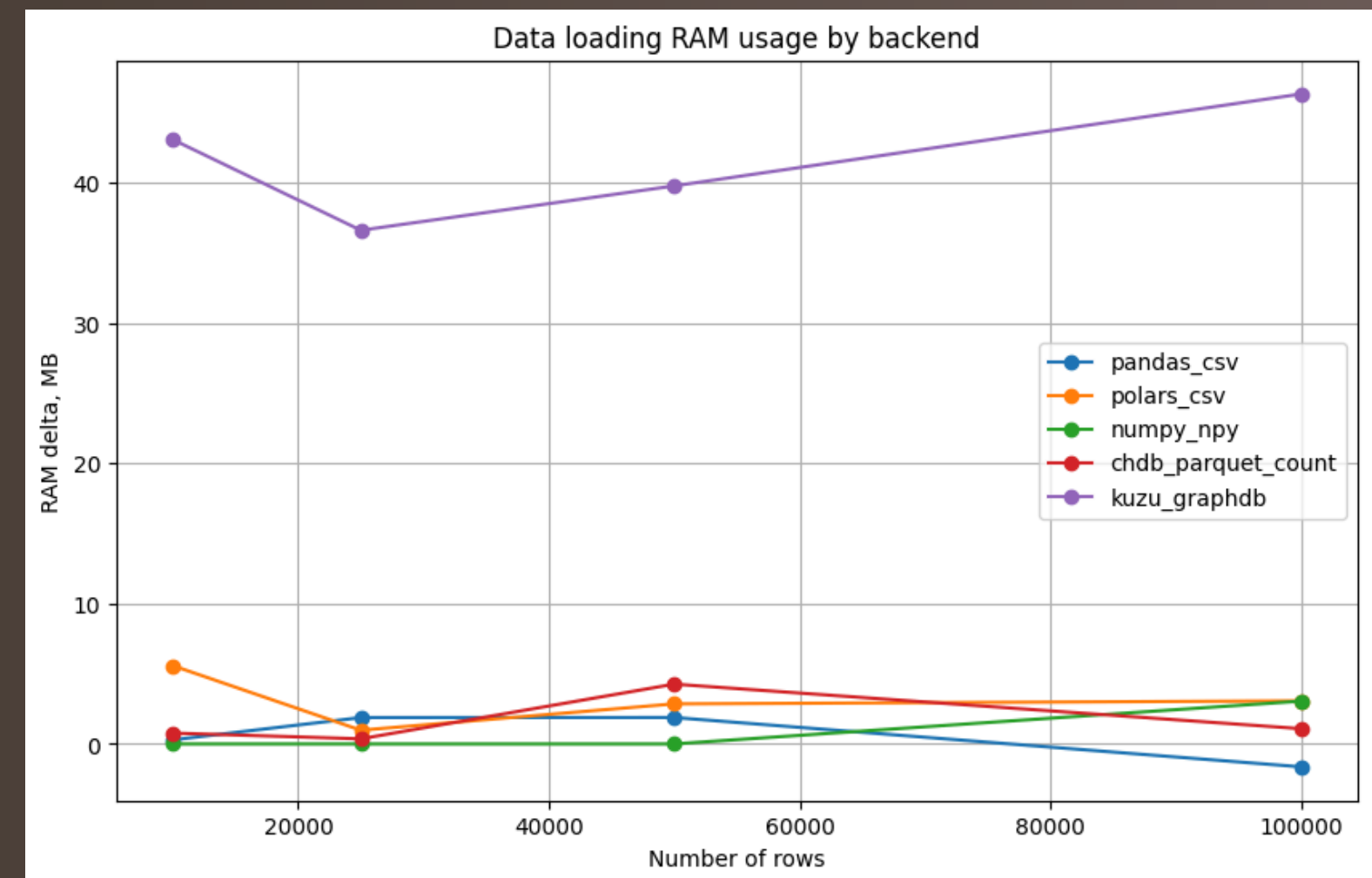
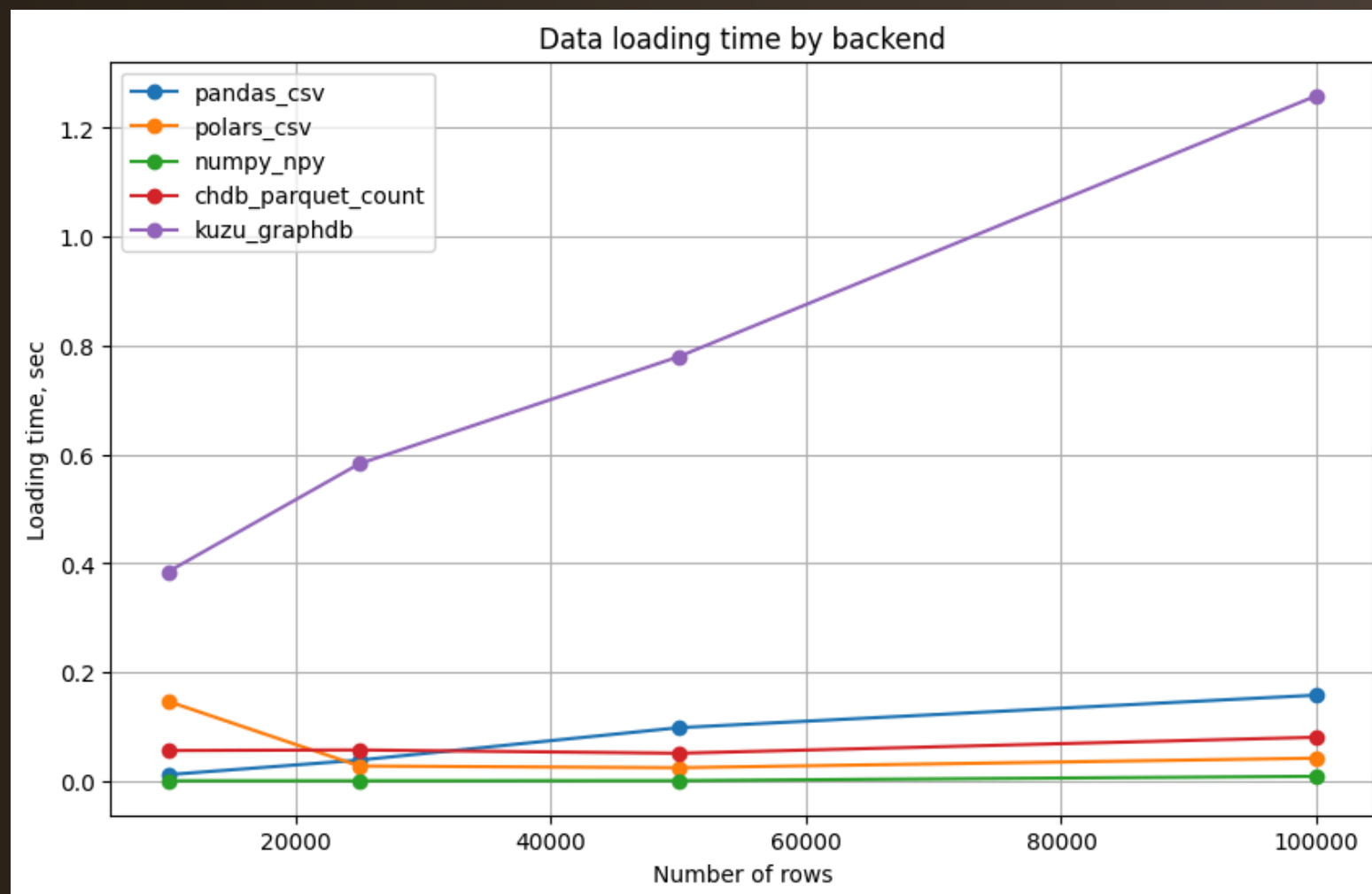
- 1.Data Loading
- 2.Filtering / querying
- 3.Aggregation
- 4.Join
- 5.Recommendation

MovieLens 20M Dataset

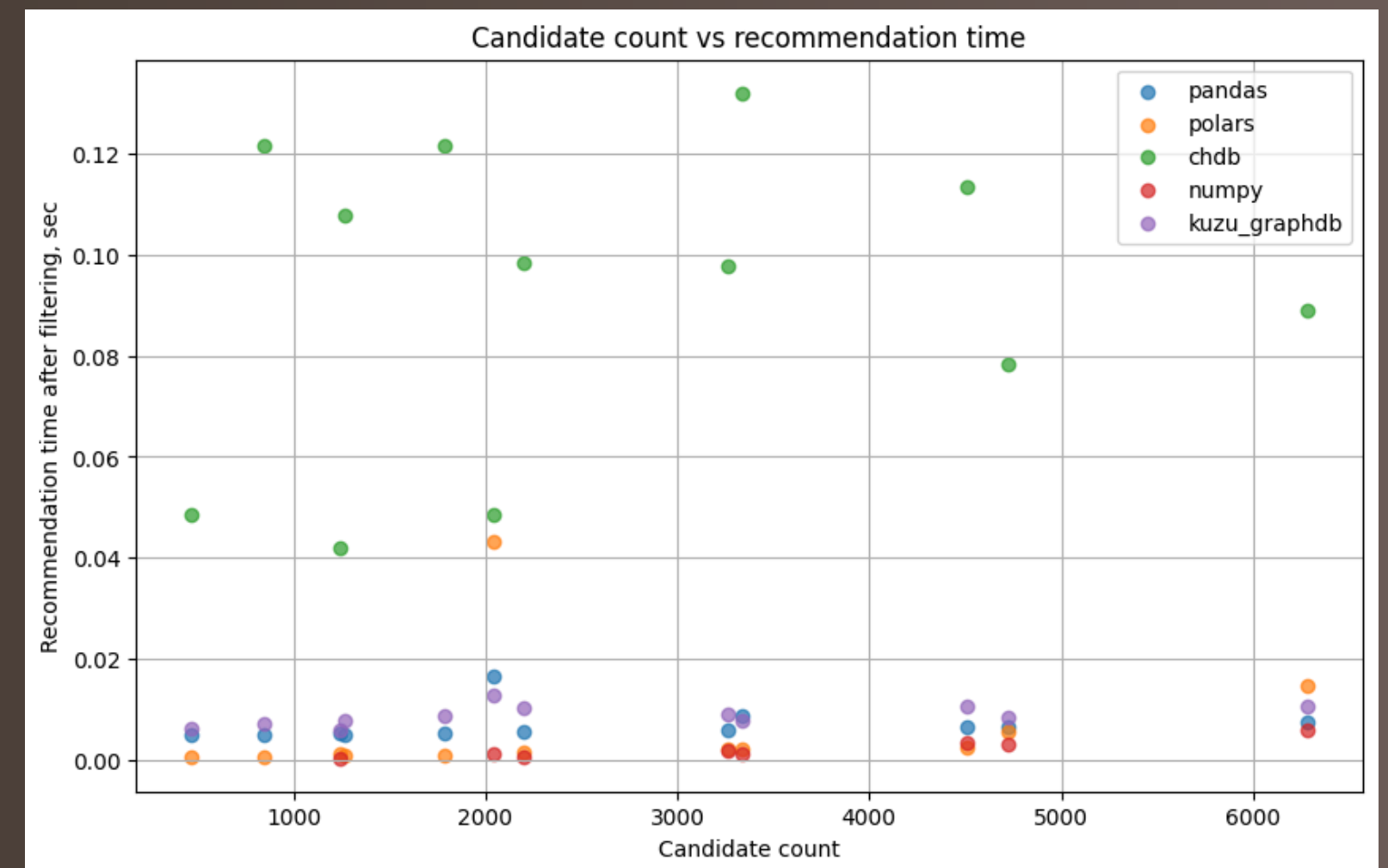
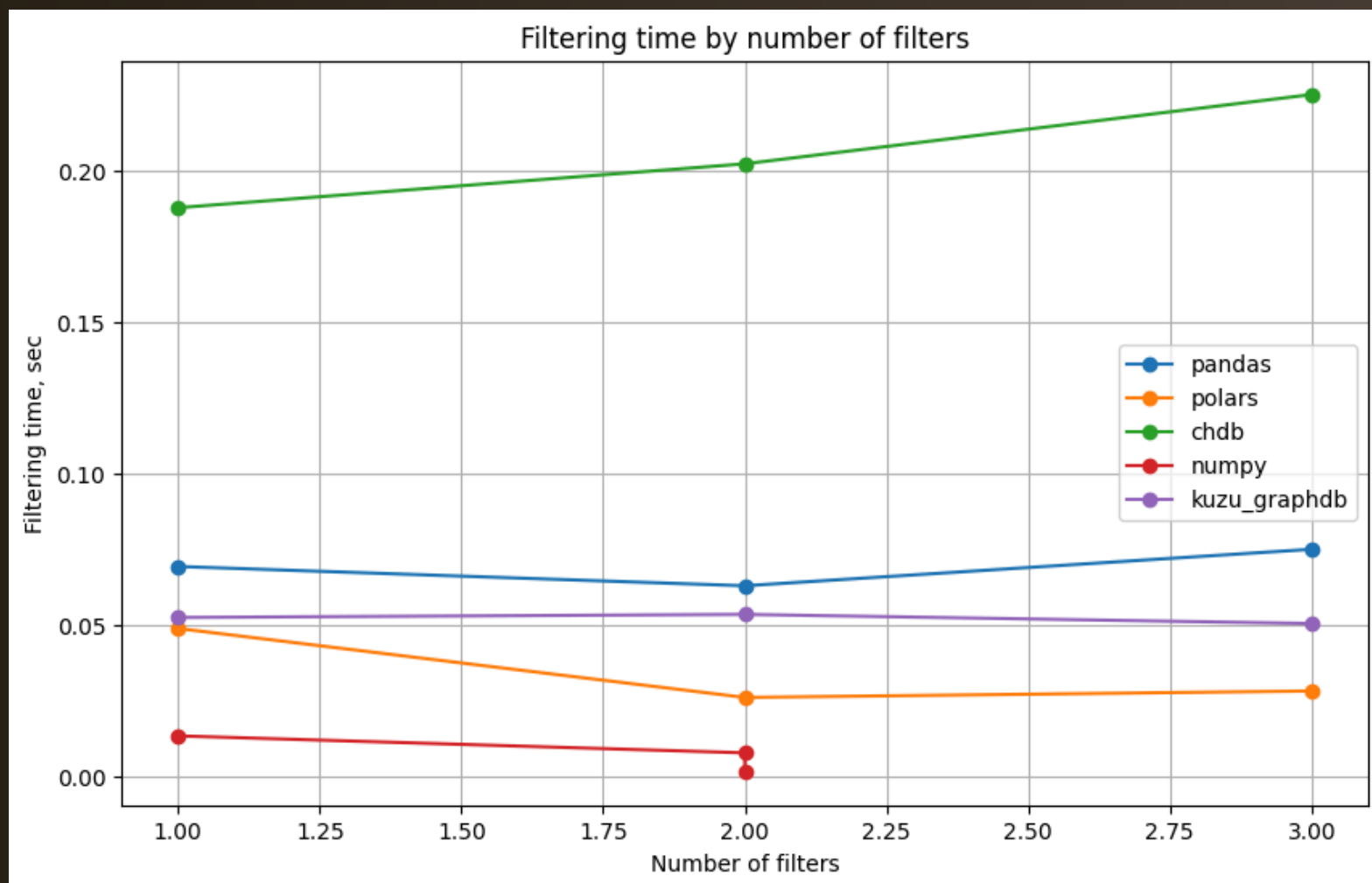
Over 20 Million Movie Ratings and Tagging Activities Since 1995

- Ratings → Edges
- Rating ≥ 4.0 → Positive interaction

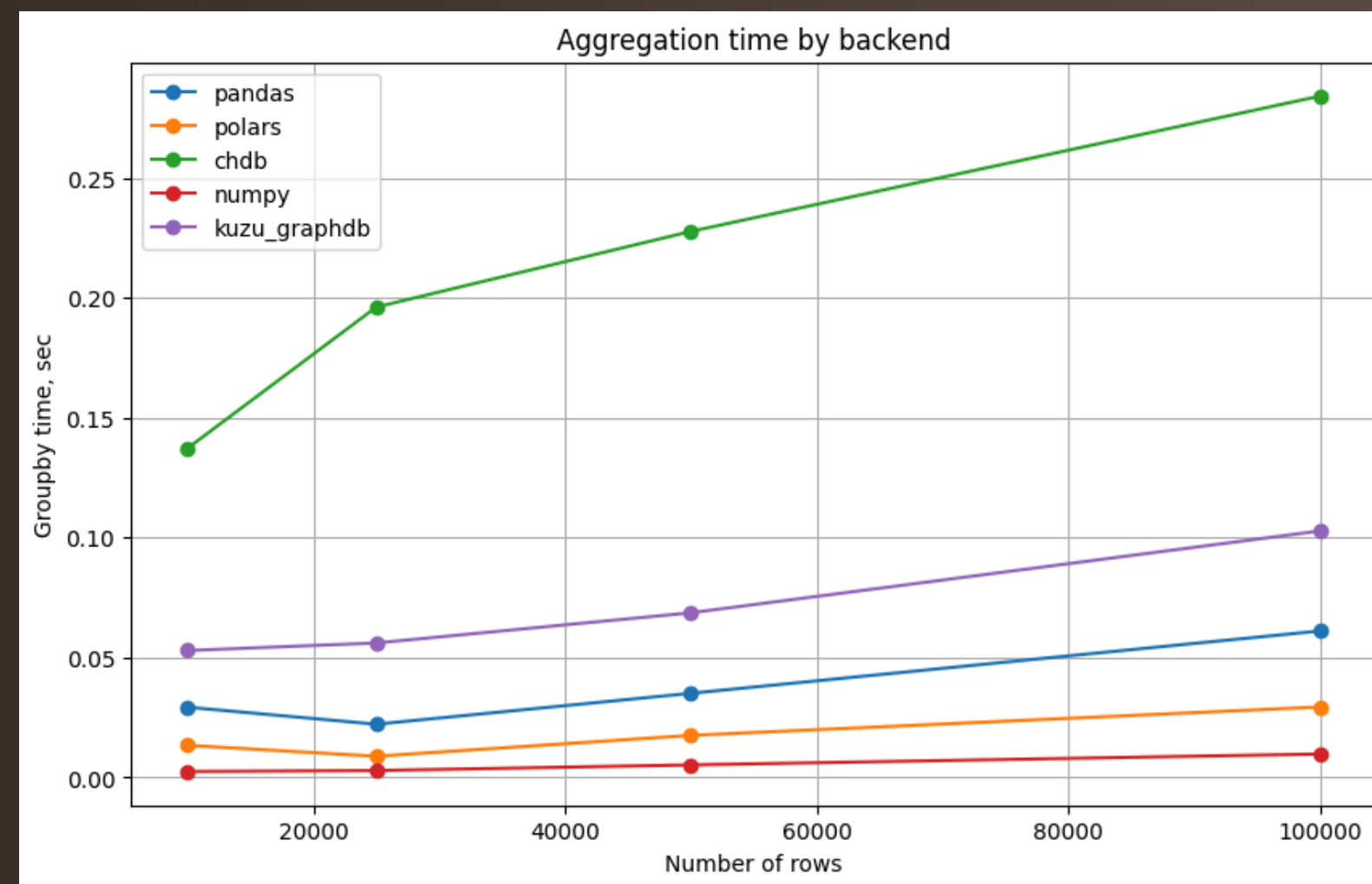
DATA LOADING



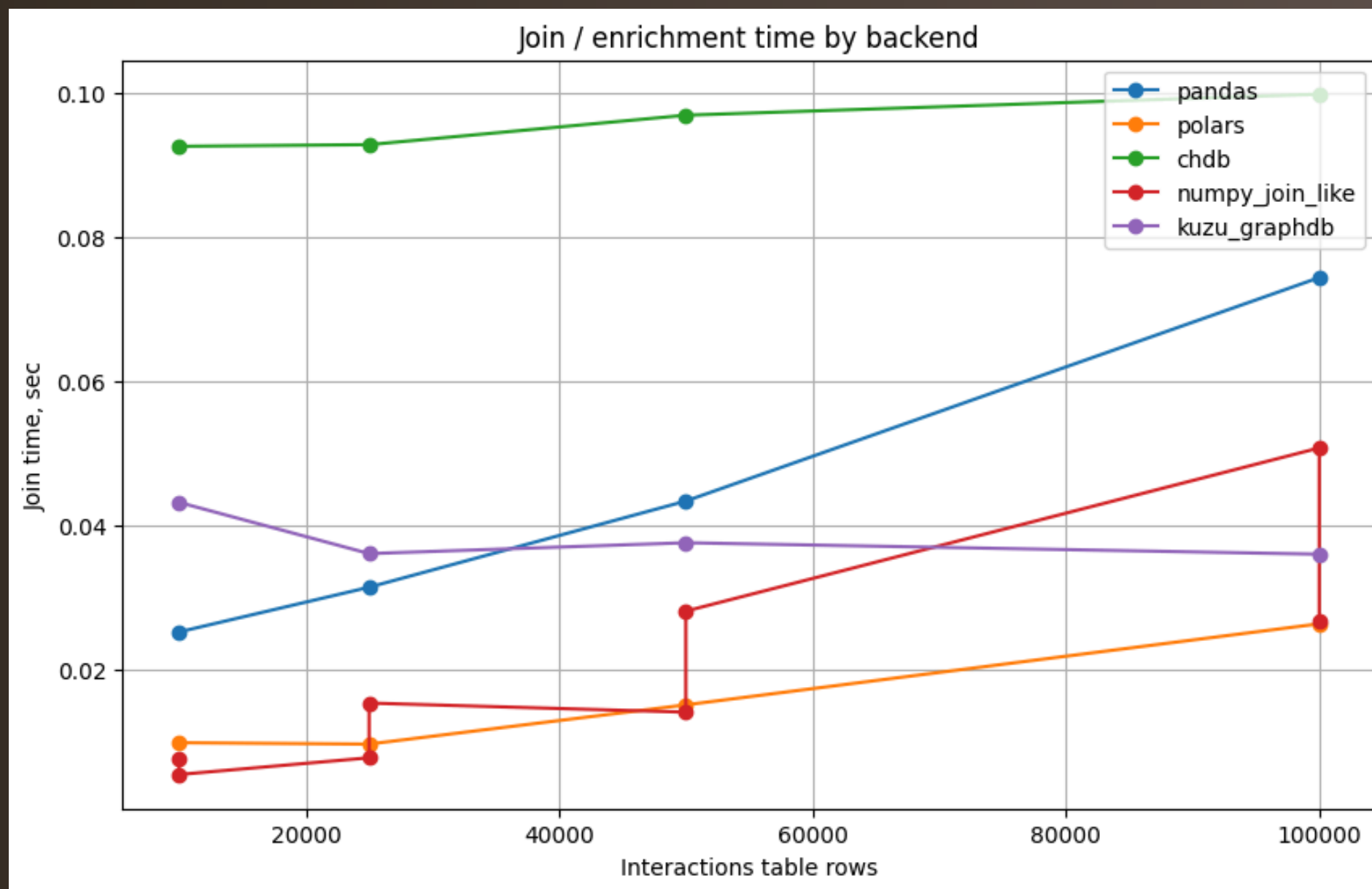
FILTERING



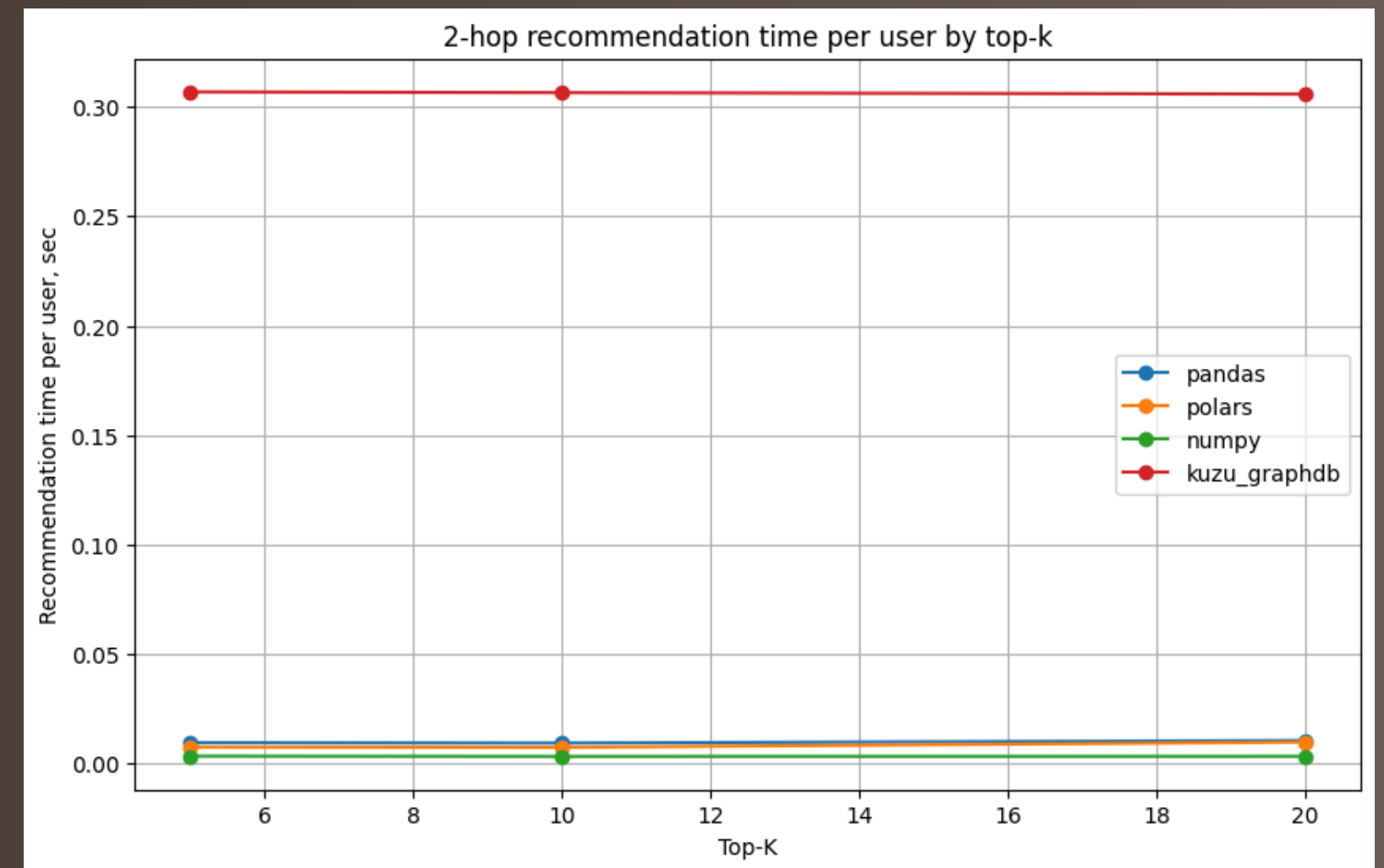
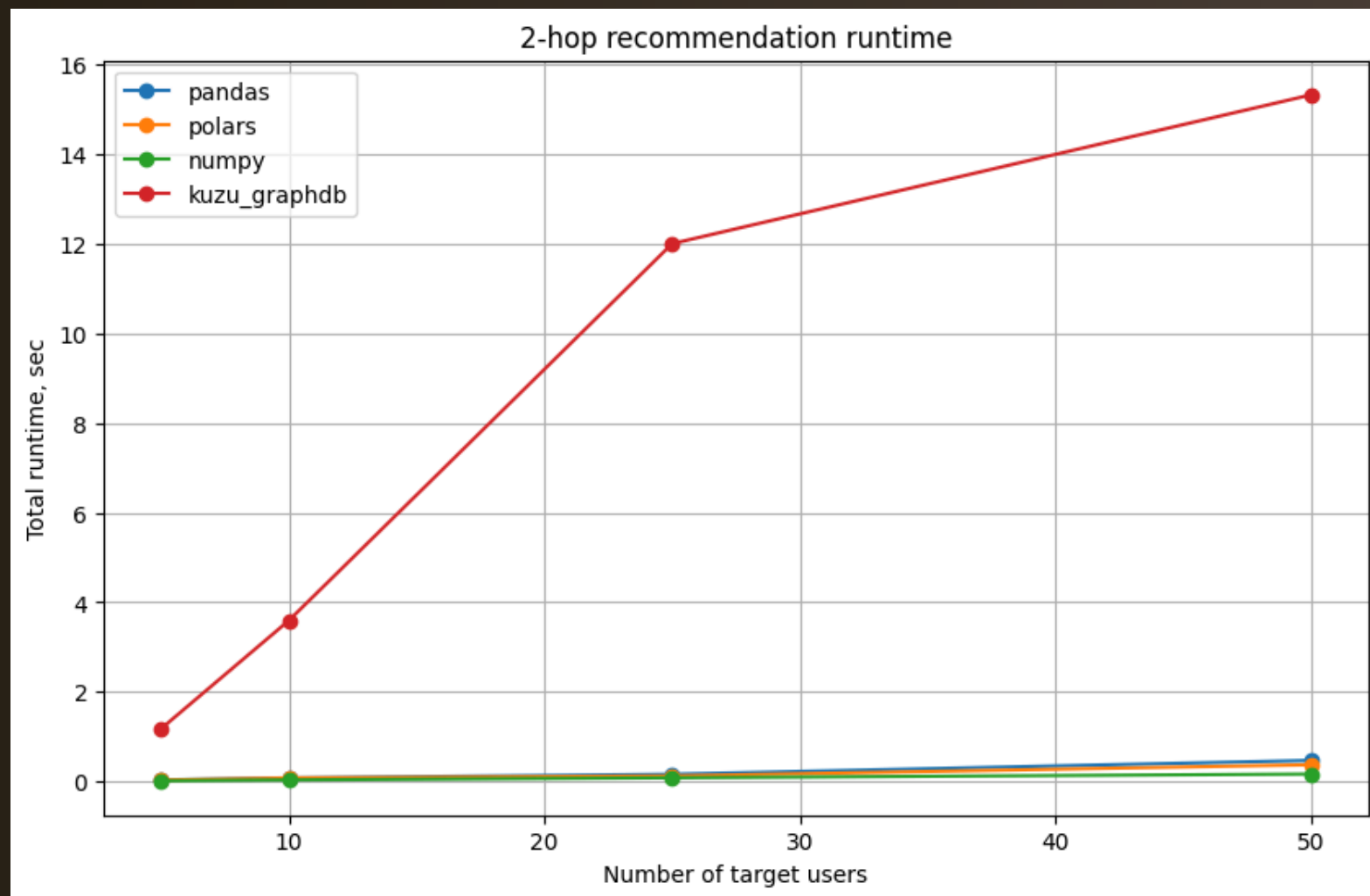
AGGREGATION



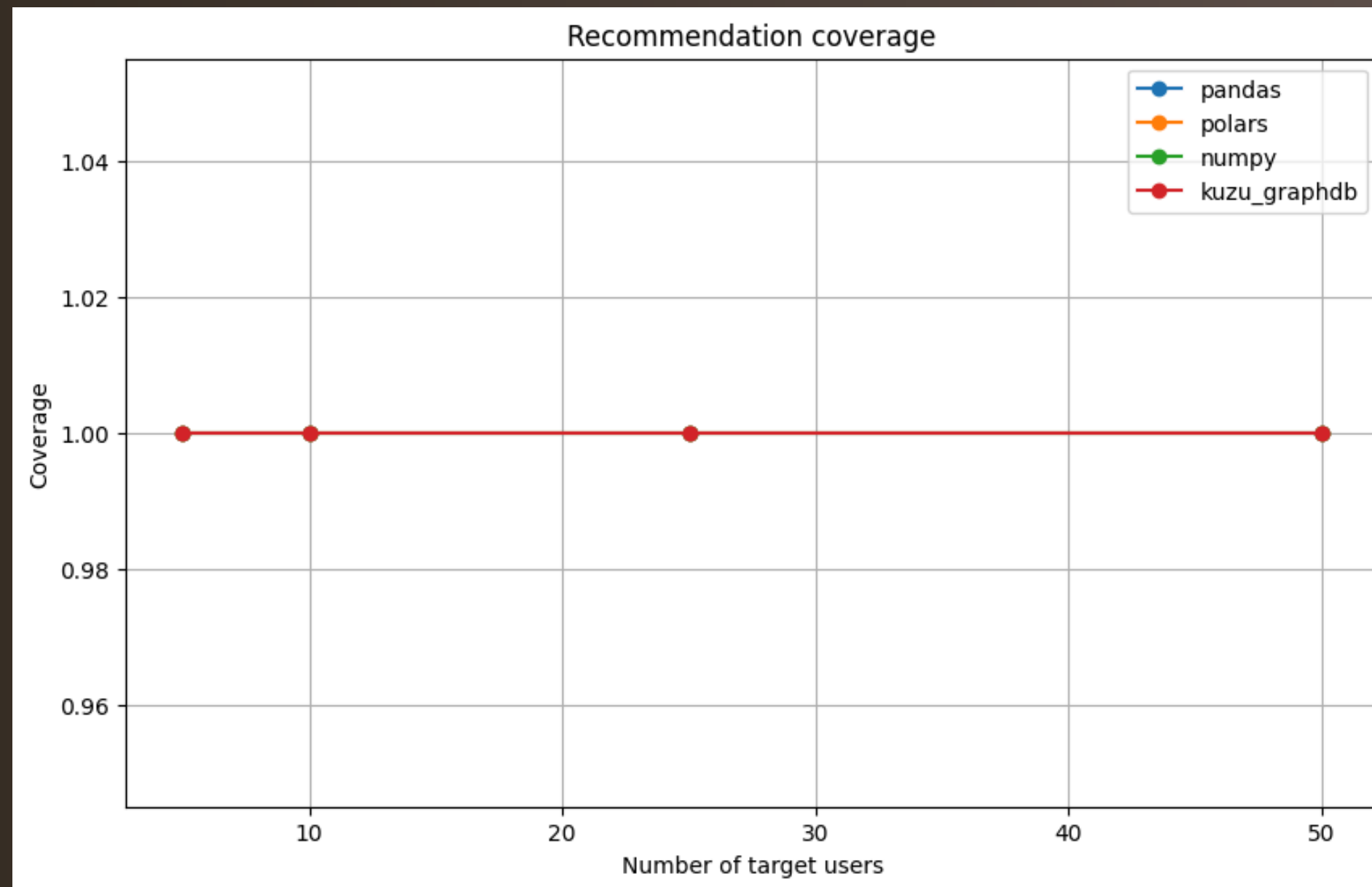
JOIN / ENRICHMENT



RECOMMENDATION



COVERAGE



BOTTLENECKS

- Data loading
- RAM delta
- NumPy join/enrichment

**THANK YOU FOR
ATTENTION**